

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi, Karnataka – 590018

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



**A DevOps Report on
Subscription Tracker-Automated Billing Reminders
deployed with Git**

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

by

Abdullah (1BY23CS002)

Aishwarya JA(1BY23CS014)

Ananya Dinesh (1BY23CS022)

Under the Guidance of

Prof. Manjula S D

Assistant Professor

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Autonomous Institute under VTU, Belagavi, Karnataka – 590019)

Yelahanka, Bengaluru, Karnataka – 560119

CERTIFICATE

This is to certify that the DevOps project entitled “**Subscription Tracker- Automated Billing Reminders deployed with Git**” is a bona fide work carried out by **Abdullah (1BY23CS002)**, **Aishwarya JA (1BY23CS014)** and **Ananya Dinesh (1BY23CS022)** of 6th semester-A section during the Academic Year 2025–26. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report.

Signature of the Guide

Prof. Manjula S D
Assistant Professor
Department of CSE

Signature of the Cluster Head

Dr. Mahesh G
Professor
Cluster 1 Head, CSE

External Viva

Sl. No.	Name of the Examiner	Signature with Date
1.		
2.		

ABSTRACT

The Subscription Tracker is a real-time, full-stack web application designed to simplify personal finance management by enabling users to track and manage their recurring subscriptions and bill payments from a single platform. With the increasing number of digital services such as streaming platforms, cloud storage, and utility services, users often struggle to remember payment due dates and monitor monthly expenses. This system eliminates manual tracking by offering a secure and centralized dashboard where users can maintain subscription details, receive reminders, and analyze their expenditure effectively.

The application is developed using Vanilla JavaScript, HTML5, and CSS3 for the frontend, ensuring a responsive and user-friendly interface, while Firebase is used as the backend service for authentication, database management, and cloud functions. Firebase Authentication provides secure access control, and Cloud Firestore stores subscription data in a structured format. Automated reminder emails are sent using SendGrid through a scheduled Firebase Cloud Function that checks upcoming due dates daily. To improve financial awareness, the system also integrates interactive expense visualization using Chart.js, allowing users to understand and control their spending behavior more efficiently.

DECLARATION

We, the undersigned, hereby declare that the project titled “**Subscription Tracker-Automated Billing Remainders deployed with Git**” is a record of web technology project work carried out under the guidance of [**Prof. Manjula S D, Assistant Professor**], Department of Computer Science and Engineering, BMS Institute of Technology & Management, an Autonomous Institute under Visvesvaraya Technological University, Belagavi, during the Academic Year 2025–2026. This work has not been submitted to any other University or Institute for the award of any degree or diploma.

Name	USN	Signature
Abdullah	1BY23CS002	
Aishwarya JA	1BY23CS014	
Ananya Dinesh	1BY23CS022	

CONTENTS

Sl. No.	Description	Page No.
1	Certificate	2
2	Abstract	3
3	Declaration	4
4	Chapter 1: Introduction	6
6	Chapter 2: Requirements Analysis	7
7	Chapter 3: System Design and Architecture	8
8	Chapter 4: Implementation	9
9	Chapter 5: Evaluation and Results	10
11	Chapter 6: Deployment and Hosting	12
12	Chapter 7: Conclusion and Future Work	16

Chapter 1: Introduction

The Subscription Tracker project addresses the growing challenge of managing multiple subscription-based services such as streaming platforms, cloud storage, utilities, and software plans. Users often struggle to remember recurring payment dates, leading to missed payments, late fees, and unnecessary expenses. The project aims to provide a centralized and automated solution that helps users track subscriptions, receive timely reminders, and manage recurring expenses more efficiently.

The system is designed to simplify subscription management through features such as secure user authentication, real-time subscription tracking, automated reminders, and expense visualization. Users can add, edit, delete, and monitor subscriptions through a responsive and user-friendly dashboard. The application also maintains secure cloud-based storage and real-time synchronization using Firebase services.

The primary objectives of the project include developing a secure subscription management platform, implementing CRUD operations, providing monthly expense analysis, and improving financial planning through automation. The system focuses on improving usability, reducing dependency on manual reminders, and helping users maintain better financial control over recurring expenses.

The project scope includes secure login functionality, cloud-based accessibility, subscription tracking, spending visualization, and automated due-date reminders. Although the current implementation does not support direct payment processing, the architecture is designed to support future integration with payment gateways and additional financial management features.

The project also contributes toward Sustainable Development Goals (SDGs) by promoting responsible financial management, encouraging the use of cloud-based digital infrastructure, and helping users reduce wasteful spending through better subscription awareness and control.

Chapter 2: Requirements Analysis

Hardware Requirements

The system is lightweight and does not require specialized hardware. The following minimum hardware configuration is sufficient for both development and usage:

- At least 4 GB RAM for smooth browser and development environment usage.
- A dual-core processor or above.
- 1 GB free disk space for source files and local testing setup.
- Stable internet connection for database access and Firebase services.

Software Requirements

The proposed system is browser-based and cloud-hosted, so it requires minimal local software:

- Operating System: Windows, Linux, or macOS.
- Internet Browser: Chrome, Firefox, Edge or Safari.
- Code Editor: Visual Studio Code or equivalent.
- Firebase Account: For authentication, cloud database, and hosting.

Survey of Web Technologies

Several alternative solutions were studied before finalizing the technology stack:

- Traditional spreadsheet-based tracking lacks automation.
- Financial apps focus on banking not subscriptions.
- Local database systems lack scalability.

Chapter 3: System Design and Architecture

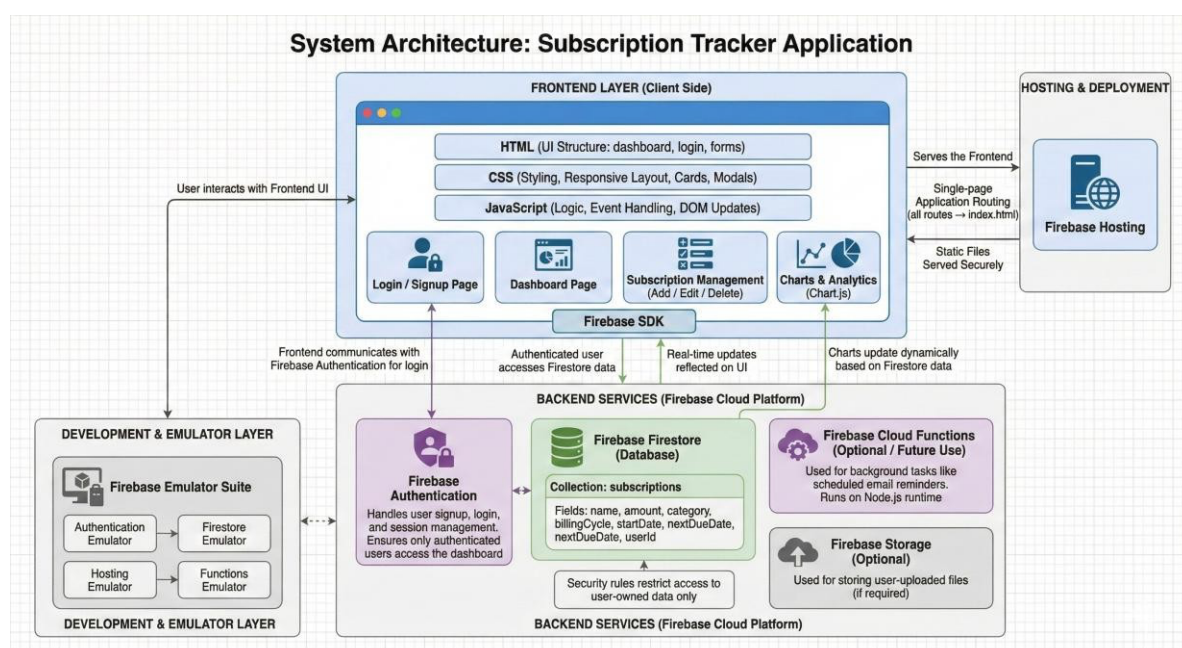
System Architecture

The Subscription Tracker application follows a modern serverless architecture that separates the frontend interface, backend services, and cloud infrastructure to improve scalability, maintainability, and security. The system uses Firebase as a Backend-as-a-Service (BaaS) platform, enabling seamless integration of authentication, database management, and hosting services without requiring traditional server management.

The frontend of the application is developed using HTML, CSS, and JavaScript, which handle user interaction, dynamic updates, and data presentation. All application logic is executed on the client side, while Firebase services manage backend functionalities such as authentication, secure data storage, and real-time synchronization. This separation keeps the frontend lightweight while ensuring backend reliability and scalability.

Instead of using a dedicated backend server, the system relies on Firebase Authentication for secure user login and Firestore for cloud-based real-time database operations. Users interact with the application through a browser, where JavaScript manages events such as form submissions, authentication, and UI updates. Firestore stores subscription records securely and synchronizes updates instantly across the application.

The architecture also supports optional Firebase Cloud Functions for server-side operations such as automated email reminders and scheduled background tasks. Overall, the serverless design reduces deployment complexity, minimizes infrastructure overhead, and provides a scalable and efficient solution for subscription management.



Chapter 4: Implementation

The Subscription Tracker follows a secure and event-driven data flow architecture where users interact with the application through browser-based forms, dashboards, and navigation components. User actions such as login, adding subscriptions, or deleting records trigger JavaScript event handlers that validate input data and communicate with Firebase services. Firebase Authentication verifies user identity, while Firestore securely processes CRUD operations and synchronizes updates instantly with the user interface. This architecture ensures real-time responsiveness, data consistency, and secure access control throughout the application. Optional Cloud Functions further support background tasks such as automated reminders and scheduled operations.

The implementation of the system relies on modern full-stack development tools and frameworks. Standard hardware such as laptops or desktop systems with internet connectivity is sufficient for development and deployment since most backend processing is handled through Firebase cloud services. The software stack includes Node.js, npm, Express.js, Firebase, Postman, and Visual Studio Code, which collectively support application development, testing, deployment, and version management. The frontend is developed using HTML5, CSS3, and JavaScript, while Firebase services provide authentication, cloud storage, hosting, and optional server-side execution through Cloud Functions.

Firebase forms the core of the implementation by enabling a serverless architecture that simplifies deployment and maintenance. Firebase Hosting is used to serve static frontend files, Firestore stores subscription-related data with real-time synchronization, and Firebase Authentication manages secure user login and access control. The Firebase Emulator Suite is used during development to safely test Authentication, Firestore, Hosting, and Cloud Functions locally without affecting production data. This setup improves development efficiency and debugging capabilities.

The deployment workflow follows a structured approach beginning with local development and testing using the Firebase Emulator Suite. After successful testing, the application is deployed using Firebase CLI commands, which upload frontend files, configure security rules, and deploy Cloud Functions when required. Overall, the Subscription Tracker demonstrates a scalable, maintainable, and cloud-native implementation that effectively combines serverless technologies, real-time databases, and responsive frontend design to deliver a reliable subscription management platform.

Chapter 5: Evaluation and Results

The Subscription Tracker system was evaluated based on functionality, usability, responsiveness, and overall performance. Functional testing confirmed that users can efficiently add, edit, delete, and manage subscriptions, with all updates reflected instantly in the interface. Features such as payment tracking, reminder notifications, filtering options, and subscription schedules operated reliably across multiple testing scenarios, demonstrating the stability and correctness of the system.

The usability and responsiveness of the application were tested across desktops, tablets, and mobile devices. The responsive interface adapts smoothly to different screen sizes, ensuring a consistent user experience across platforms. Navigation elements, subscription lists, and dashboard features are designed to be intuitive and easily accessible, allowing users to perform actions such as managing subscriptions or viewing reminders without confusion.

The results of the implementation demonstrate that the Subscription Tracker successfully fulfills its objective of providing a centralized and user-friendly subscription management platform. The integration of frontend technologies with Firebase services ensures smooth communication, secure authentication, and real-time data synchronization. The system remains responsive even during multiple user interactions, highlighting the effectiveness of the selected architecture and technology stack.

From a performance perspective, Firestore enables efficient real-time data retrieval and synchronization with low latency, while the lightweight frontend ensures smooth rendering and interaction. The modular system design also improves scalability and extensibility, allowing future enhancements such as advanced analytics, notification improvements, and additional financial management features to be integrated with minimal architectural changes.

The Subscription Tracker system was evaluated based on functionality, usability, responsiveness, and overall performance. Functional testing confirmed that users can efficiently add, edit, delete, and manage subscriptions, with all updates reflected instantly in the interface. Features such as payment tracking, reminder notifications, filtering options, and subscription schedules operated reliably across multiple testing scenarios, demonstrating the stability and correctness of the system.

The usability and responsiveness of the application were tested across desktops, tablets, and mobile devices. The responsive interface adapts smoothly to different screen sizes, ensuring a consistent user experience across platforms. Navigation elements, subscription lists, and dashboard features are designed to be intuitive and easily accessible, allowing users to perform actions such as managing subscriptions or viewing reminders without confusion.

The results of the implementation demonstrate that the Subscription Tracker successfully fulfills its objective of providing a centralized and user-friendly subscription management platform. The integration of frontend technologies with Firebase services ensures smooth communication, secure authentication, and real-time data synchronization. The system

remains responsive even during multiple user interactions, highlighting the effectiveness of the selected architecture and technology stack.

From a performance perspective, Firestore enables efficient real-time data retrieval and synchronization with low latency, while the lightweight frontend ensures smooth rendering and interaction. The modular system design also improves scalability and extensibility, allowing future enhancements such as advanced analytics, notification improvements, and additional financial management features to be integrated with minimal architectural changes.

Your Subscriptions

Subscription Name	Category	Cost (₹)	Next Billing Date	Action
Netflix	Entertainment	₹499.00	Next: 1 Jan 2026	Edit +
Ebook	Education	₹99.00	Next: 18 Jan 2026	Edit +
Spotify	Music	₹299.00	Next: 31 Jan 2026	Edit +
Library	Education	₹1999.00	Next: 15 Sept 2026	Edit +

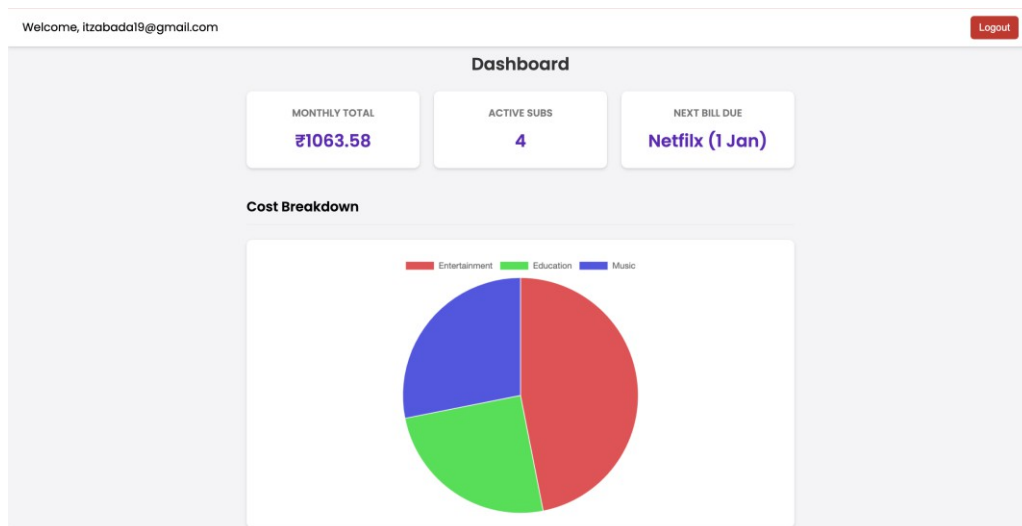
Add New Subscription

Subscription Name: Cost (₹):

Category: Billing Cycle:

First Billing Date:

Add Subscription



Chapter 6: Deployment and Hosting

The Subscription Tracker application was deployed using GitHub Pages to make the system publicly accessible through a secure and reliable cloud-hosted environment. The deployment process converts the locally developed application into a production-ready web application that can be accessed from anywhere using a browser. The deployed application is available at <https://abdullah-habeeb.github.io/subscription-tracker>

Why Git Was Selected

- **Simple and Easy Deployment**
GitHub Pages allows direct deployment from a GitHub repository with minimal setup and configuration.
- **Lightweight Hosting Solution**
It is ideal for static frontend applications built using HTML, CSS, and JavaScript without requiring dedicated servers.
- **Reduced Infrastructure Complexity**
Unlike Docker and Jenkins, GitHub Pages does not require container management, pipeline setup, or server maintenance.
- **Integrated Version Control**
GitHub automatically tracks all code changes, enabling efficient version management and rollback support.
- **Automatic Hosting Support**
Frontend files are hosted directly from the selected repository branch without additional deployment scripts.
- **Faster Development Workflow**
Updates pushed to the GitHub repository are reflected quickly in the deployed application, simplifying testing and maintenance.
- **Cost-Effective Deployment**
GitHub Pages provides free hosting for static applications, making it suitable for academic and prototype projects.
- **Better Suitability for Static Applications**
The Subscription Tracker frontend does not require backend microservices, container orchestration, or advanced CI/CD pipelines.
- **Avoids Unnecessary Docker Complexity**
Docker is mainly beneficial for applications requiring isolated runtime environments and containerized backend services.
- **Avoids Jenkins Pipeline Overhead**
Jenkins is more suitable for enterprise-scale automation and continuous integration workflows, which were unnecessary for this project.
- **Improved Maintainability**
The deployment process remains simple, organized, and easier to manage for small- to medium-scale applications.
- **Efficient Frontend Hosting**
GitHub Pages efficiently serves static assets such as HTML, CSS, and JavaScript files with reliable accessibility.

Deployment Architecture

The deployment architecture consists of the following components:

- GitHub Repository for source code management.
- GitHub Pages for frontend hosting.
- Firebase services for backend functionalities such as Authentication and Firestore.
- Browser-based client interface for user interaction.

The frontend application communicates directly with Firebase services using Firebase SDKs. GitHub Pages only hosts the frontend files, while all backend operations such as authentication, database storage, and synchronization are handled by Firebase cloud infrastructure.

Deployment Workflow

The deployment process followed a structured workflow to ensure reliability and consistency between development and production environments.

Step 1: Local Development

The application frontend was developed locally using:

- HTML5
- CSS3
- JavaScript
- Firebase SDK integration

All functionalities such as authentication, CRUD operations, reminders, and dashboard rendering were tested locally before deployment.

Step 2: Git Repository Initialization

A Git repository was initialized to maintain version control and track project changes.

Example commands used:

```
git init
```

```
git add .
```

```
git commit -m "Initial project deployment"
```

This ensured proper version tracking and collaborative development support.

Step 3: Creating GitHub Repository

A new repository named subscription-tracker was created on GitHub. The local repository was then connected to the remote GitHub repository.

Example commands:

```
git remote add origin https://github.com/abdullah-habeeb/subscription-tracker.git
```

```
git branch -M main
```

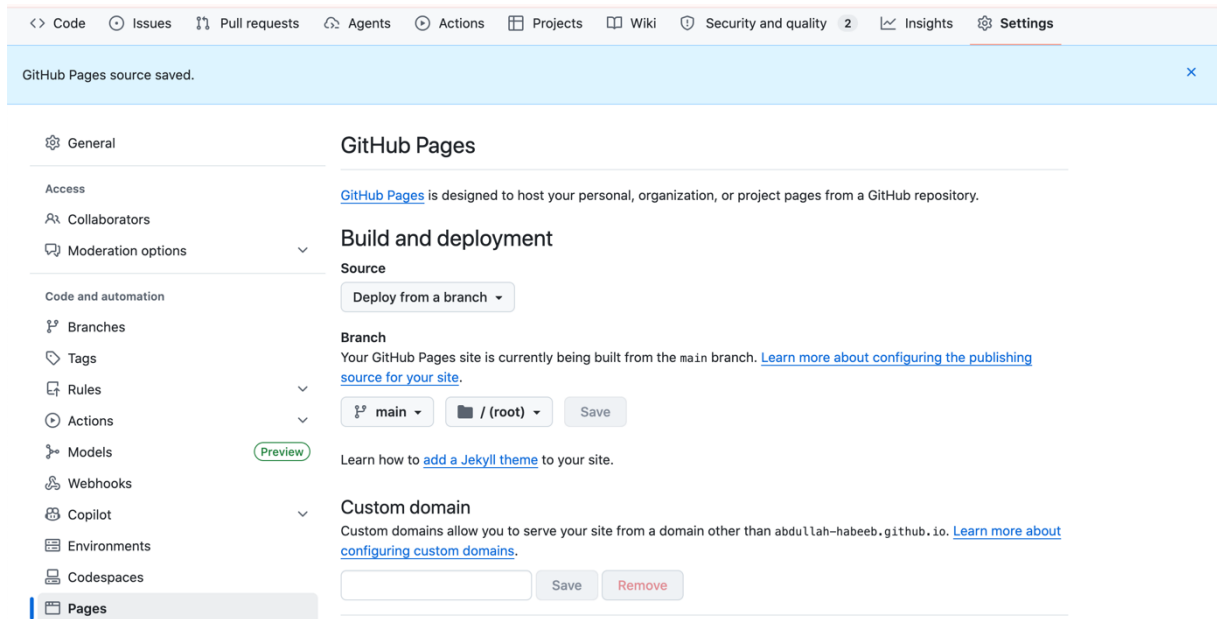
```
git push -u origin main
```

Step 4: Enabling GitHub Pages

GitHub Pages was enabled from the repository settings by:

1. Opening the repository settings.
2. Navigating to the “Pages” section.
3. Selecting the deployment branch as main.
4. Selecting the root folder for hosting.
5. Saving the configuration.

After configuration, GitHub automatically generated the hosted deployment URL.



Step 5: Production Deployment

Once GitHub Pages was enabled:

- Static frontend files were hosted automatically.
- Application assets became publicly accessible.
- Deployment updates occurred automatically whenever new commits were pushed.
-

Step 6: Deployment Verification

The deployed application was tested for:

- Proper routing and navigation
- Firebase authentication functionality
- Firestore database operations
- Responsive UI behavior
- Cross-browser compatibility

All core functionalities operated successfully in the deployed environment.

Features of the Deployed Application

- Secure user authentication using Firebase Authentication.
- User registration and login functionality.
- Dynamic session management for authenticated users.
- Ability to add new subscriptions.
- Ability to edit existing subscription details.
- Ability to delete subscriptions.
- Viewing recurring payment schedules and subscription records.
- Real-time database synchronization using Firestore.
- Instant data updates without manual page refresh.
- Interactive expense visualization through charts and dashboards.

- Responsive user interface compatible with desktops, tablets, and mobile devices.
- Cloud-based accessibility from any internet-enabled device.
- Lightweight and user-friendly frontend interface.
- Secure storage and management of user subscription data.
- Real-time dashboard updates for improved user experience.

Advantages of the Deployment Model

- Lightweight cloud hosting using GitHub Pages.
- Simplified deployment and hosting management.
- Automatic deployment updates through Git-based workflows.
- Integrated version control and source code management.
- Improved maintainability and scalability.
- Public accessibility through a hosted deployment URL.
- Reduced infrastructure and operational overhead.
- Faster deployment and update cycles.
- Reliable frontend hosting for static web applications.
- Easy collaboration among developers using Git repositories.
- Efficient separation between frontend and backend services.
- Practical implementation of serverless architecture concepts.
- Support for continuous deployment workflows.
- Improved application availability and accessibility.
- Cost-effective deployment solution suitable for academic and small-scale projects.

Overall, the deployment of the Subscription Tracker using GitHub Pages successfully demonstrates how modern frontend applications can be efficiently hosted and managed using lightweight cloud deployment platforms while maintaining scalability, accessibility, and reliability.

abduallah-habeeb.github.io/subscription-tracker/index.html

WhatsApp Contineo AWS Skill Builder BMSIT - Google Dr... All Bookmarks

Welcome, abdullahhh1426@gmail.com Logout

Subscription Name	Cost (₹)	Category	Billing Cycle	Next Billing Date
Netflix	₹499.00	Entertainment	Monthly	Next: 21 May 2026
Spotify	₹99.00	Music	Monthly	Next: 25 May 2026

Add New Subscription

Subscription Name:

Cost (₹):

Category:

Billing Cycle:

First Billing Date:

Add Subscription

Chapter 7: Conclusion and Future Work

The Subscription Tracker project successfully demonstrates the development of a modern and user-friendly web application for managing recurring subscriptions and expenses. The system centralizes subscription tracking, reduces the chances of missed payments, and helps users monitor their recurring financial commitments more effectively. By automating reminders and subscription monitoring, the application minimizes manual effort while improving reliability and financial awareness.

The project also emphasizes security, usability, and scalability. Firebase Authentication and secure access controls protect user data, while the responsive interface ensures smooth usability across desktops, tablets, and mobile devices. The use of modern frontend technologies and cloud-based infrastructure enables efficient performance, maintainability, and simplified deployment.

From a technical perspective, the project applies important software engineering concepts such as modular design, serverless architecture, scalability, and cloud integration. The implementation demonstrates how full-stack development technologies and Firebase services can be combined to create a reliable and efficient subscription management platform.

Several future enhancements can further improve the system's functionality and user experience. Planned improvements include payment gateway integration, automated subscription detection using machine learning, multi-device synchronization, advanced analytics, budget planning features, calendar integration, and mobile application support. These enhancements highlight the extensibility of the system and its potential to evolve into a comprehensive personal finance management solution suitable for both individuals and organizations.

